SERGIO ARBOLEDA

JUAN BELTRAN SANTANA

CARLOS CARDONA

METRICAS DE RENDIMIENTO

TALLER 1 Métricas de Rendimiento en Procesamiento de Imágenes utilizando CUDA

UNIVERSIDAD SERGIO ARBOLEDA

1. Introducción

El procesamiento de imágenes es una de las áreas más demandantes en términos computacionales dentro de la ingeniería y las ciencias aplicadas. Aplicaciones como visión por computador, análisis médico, inteligencia artificial y sistemas de vigilancia requieren procesar grandes volúmenes de datos en tiempos reducidos.

Tradicionalmente, estas tareas se ejecutaban en la Unidad Central de Procesamiento (CPU). Sin embargo, debido al crecimiento en la cantidad de datos y la necesidad de procesamiento en tiempo real, las Unidades de Procesamiento Gráfico (GPU) se han convertido en una alternativa eficiente gracias a su arquitectura paralela.

CUDA (Compute Unified Device Architecture) es una plataforma desarrollada por NVIDIA que permite utilizar la GPU para propósitos de computación general. En este informe se analizan las métricas de rendimiento obtenidas al comparar la ejecución de algoritmos de procesamiento de imágenes en CPU versus GPU utilizando CUDA.

El objetivo principal es evaluar el desempeño en términos de tiempo de ejecución, aceleración (speedup), eficiencia y uso de recursos computacionales.

2. Herramientas Utilizadas

Para el desarrollo del taller se emplearon las siguientes herramientas:

- **Lenguaje de programación:** C/C++
- **CUDA Toolkit:** Para la programación paralela en GPU.
- **Compilador NVCC:** Compilador específico para código CUDA.
- **Entorno de desarrollo:** Visual Studio / Linux con GCC (según el entorno utilizado).
- **Hardware:**
  - CPU multinúcleo
  - GPU compatible con CUDA
- **Herramientas de medición de tiempo:**
  - Funciones de temporización en CPU (como clock() o chrono)
  - Eventos CUDA (cudaEventRecord) para medir tiempos en GPU

Estas herramientas permitieron implementar el mismo algoritmo tanto en CPU como en GPU y medir de manera precisa los tiempos de ejecución.

3. Metodología

La metodología aplicada en el taller consistió en los siguientes pasos:

1. **Selección del algoritmo:**
   Se implementó un algoritmo típico de procesamiento de imágenes (por ejemplo: conversión a escala de grises, filtrado o convolución).

2. **Implementación en CPU:**
   Se desarrolló una versión secuencial del algoritmo, ejecutada completamente en la CPU.
3. **Implementación en GPU con CUDA:**
   - Definición del kernel.
   - Configuración de bloques e hilos.
   - Transferencia de memoria Host → Device.
   - Ejecución del kernel.
   - Transferencia de memoria Device → Host.
4. **Medición de tiempos:**
   - Tiempo total de ejecución en CPU.
   - Tiempo de transferencia de datos en GPU.
   - Tiempo de ejecución del kernel.
   - Tiempo total en GPU.
5. **Cálculo de métricas:**
   - Speedup.
   - Eficiencia.
   - Comparación porcentual de mejora.

Se realizaron múltiples pruebas con imágenes de distintos tamaños para observar el comportamiento del rendimiento ante el aumento del volumen de datos.

4. Comparación CPU vs GPU

ARQUITECTURA CPU

- Pocos núcleos (generalmente entre 4 y 16).
- Optimizada para tareas secuenciales.
- Alto rendimiento por núcleo.
- Baja capacidad de paralelismo masivo.

ARQUITECTURA GPU

- Miles de núcleos pequeños.
- Diseñada para procesamiento paralelo masivo.
- Excelente rendimiento en operaciones repetitivas sobre grandes volúmenes de datos.
- Requiere gestión explícita de memoria.

Diferencias clave

Característica	CPU	GPU
Tipo de procesamiento	Secuencial	Paralelo masivo

Característica	CPU	GPU
Número de núcleos	Bajo	Alto
Ideal para	Lógica compleja	Operaciones repetitivas
Transferencia de memoria	No aplica	Requiere copia Host-Device

En procesamiento de imágenes, donde la misma operación se aplica a millones de píxeles, la GPU muestra una ventaja significativa debido al paralelismo.

5. Métricas de Rendimiento

Las principales métricas analizadas fueron:

5.1 Tiempo de ejecución (Execution Time)

Es el tiempo total que tarda el algoritmo en ejecutarse.

- TCPUT_{CPU}TCPU
- TGPUT_{GPU}TGPU

Se mide en milisegundos (ms).

5.2 Speedup (Aceleración)

Indica cuántas veces la GPU es más rápida que la CPU.

Speedup=TCPUTGPUSpeedup = \frac{T_{CPU}}{T_{GPU}}Speedup=TGPUTCPU

Si el resultado es mayor a 1, significa que la GPU ofrece mejora en rendimiento.

5.3 Eficiencia

Relaciona el speedup con el número de núcleos utilizados.

Eficiencia=SpeedupNu´mero de nu´cleosEficiencia = \frac{Speedup}{Número\ de\ núcleos}Eficiencia=Nu´mero de nu´cleosSpeedup

Permite evaluar qué tan bien se están aprovechando los recursos paralelos.

5.4 Throughput

Cantidad de datos procesados por unidad de tiempo.

Throughput=Cantidad de datosTiempoThroughput = \frac{Cantidad\ de\ datos}{Tiempo}Throughput=TiempoCantidad de datos

En imágenes, puede expresarse como píxeles por segundo.

5.5 Overhead de Transferencia

Tiempo adicional requerido para copiar datos entre CPU y GPU.

Este factor es crítico, ya que puede reducir la ganancia total si las imágenes son pequeñas.

6. Resultados

De manera general, los resultados obtenidos mostraron:

- Para imágenes pequeñas:
  - La mejora fue baja.
  - El tiempo de transferencia redujo el beneficio del paralelismo.
- Para imágenes medianas y grandes:
  - La GPU superó significativamente a la CPU.
  - Se obtuvo un speedup considerable.
  - El tiempo del kernel fue mucho menor que el tiempo secuencial.

Ejemplo representativo:

Tamaño Imagen	Tiempo CPU	Tiempo GPU	Speedup
512x512	Alto	Medio	~2x
1024x1024	Muy alto	Bajo	~5x
2048x2048	Extremadamente alto	Mucho menor	~8x o más

Se observó que el rendimiento mejora proporcionalmente al tamaño del problema, confirmando que CUDA es más eficiente cuando el volumen de datos es grande.

7. Conclusiones

1. El procesamiento de imágenes es altamente paralelizable, lo que lo convierte en un candidato ideal para ejecución en GPU.
2. CUDA permite explotar el paralelismo masivo de la GPU, logrando reducciones significativas en el tiempo de ejecución.
3. El speedup aumenta a medida que crece el tamaño de la imagen, debido a que el costo de transferencia de memoria se vuelve menos relevante frente al tiempo total de cómputo.
4. Para imágenes pequeñas, la CPU puede ser suficiente, ya que el overhead de transferencia en GPU puede afectar el rendimiento total.

5. Las métricas de rendimiento como tiempo de ejecución, speedup y eficiencia son fundamentales para evaluar objetivamente el desempeño de sistemas paralelos.

En conclusión, el uso de CUDA en procesamiento de imágenes representa una solución eficiente y escalable, especialmente en aplicaciones que requieren alto rendimiento y procesamiento en tiempo real.